Math Module

The math module in Python provides access to mathematical functions defined by the C standard. It is widely used for numeric and scientific computations.

import math

1. Basic Arithmetic Functions

math.ceil(x)
Description: Returns the smallest integer greater than or equal to x.
Example:
math.ceil(4.3) # Output: 5

math.floor(x)
Description: Returns the largest integer less than or equal to x.
Example:
math.floor(4.7) # Output: 4

math.trunc(x)
Description: Truncates the decimal and returns the integer part.
Example:
math.trunc(-4.9) # Output: -4

2. Power and Logarithmic Functions
math.pow(x, y)
Description: Returns x raised to the power y (i
Example:
math.pow(2, 3) # Output: 8.0

math.sqrt(x)
Description: Returns the square root of x.
Example:
math.sqrt(25) # Output: 5.0

math.exp(x)
Description: Returns where e is Euler's number (~2.718).
Example:
math.exp(1) # Output: 2.718281828459045

math.log(x, base)
Description: Returns the logarithm of x to the specified base.
Default Base: e if not provided.

Example:
math.log(100, 10) # Output: 2.0
math.log(2.718281828459045) # Output: 1.0 (log base e)

math.log10(x)
Description: Returns the base-10 logarithm of x.
Example:
math.log10(1000) # Output: 3.0

3. Trigonometric Functions
These functions assume the input in radians.

math.sin(x)
Description: Returns the sine of x radians.
Example:
math.sin(math.pi / 2) # Output: 1.0

math.cos(x)
Description: Returns the cosine of x radians.
Example:
math.cos(0) # Output: 1.0

math.tan(x)
Description: Returns the tangent of x radians.
Example:
math.tan(math.pi / 4) # Output: ~1.0

math.asin(x), math.acos(x), math.atan(x)
Description: Inverse trigonometric functions. Return angle in radians.
Example:
math.asin(1) # Output: π/2

4. Angle Conversion
math.degrees(x)
Description: Converts angle from radians to degrees.
Example:
math.degrees(math.pi) # Output: 180.0

math.radians(x)
Description: Converts angle from degrees to radians.
Example:
math.radians(180) # Output: π

5. Special Mathematical Constants

math.pi
Description: Mathematical constant π = 3.14159…
Example:
math.pi # Output: 3.141592653589793

math.e
Description: Euler's number e = 2.71828…
Example:
math.e # Output: 2.718281828459045

Random module

The random module provides functions to perform random operations like generating random numbers, selecting random elements, shuffling data, and more.

import random

1. Random Number Generation

random.random()
Description: Returns a random float number between 0.0 (inclusive) and 1.0 (exclusive).
Example:
random.random() # Output: e.g., 0.6849

random.uniform(a, b)
Description: Returns a random float number between a and b.
Example:
random.uniform(10, 20) # Output: e.g., 14.726

random.randint(a, b)
Description: Returns a random integer between a and b (inclusive).
Example:
random.randint(1, 6) # Output: e.g., 4

random.randrange(start, stop[, step])
Description: Returns a randomly selected element from the range [start, stop) with an optional step.
Example:

random.randrange(0, 10, 2) # Output: e.g., 6

2. Random Choices from Sequences

random.choice(sequence)
Description: Returns a randomly selected element from a non-empty sequence (like list, string, tuple).
Example:
random.choice(['apple', 'banana', 'cherry']) # Output: e.g., 'banana'

random.choices(population, weights=None, k=1)
Description: Returns a list of k elements chosen from the population with replacement. Optional weights for probability.
Example:
random.choices(['red', 'green', 'blue'], k=2) # Output: e.g., ['green', 'green']

random.sample(population, k)
Description: Returns a list of k unique elements chosen from the population without replacement.
Example:
random.sample(range(1, 50), 6) # Output: e.g., [5, 12, 27, 33, 41, 46]

3. Shuffling Data
random.shuffle(sequence)
Description: Shuffles the elements of a list in place (modifies the original list).
Example:
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)
print(cards) # Output: e.g., [3, 1, 5, 4, 2]

4. Setting the Random Seed
random.seed(a=None)
Description: Initializes the random number generator. Useful for reproducibility.
Example:
random.seed(10)
print(random.random()) # Always gives the same output for seed=10

OS module

The os module provides a way of using operating system-dependent functionality, such as reading or writing to the file system, managing directories, and environment variables.

import os

1. Directory and File Management

os.getcwd()
Description: Returns the current working directory.
Example:
os.getcwd() # Output: e.g., '/home/user/project'

os.chdir(path)
Description: Changes the current working directory to path.
Example:
os.chdir('/home/user/Documents')

os.listdir(path='.')
Description: Returns a list of files and directories in the given path. Defaults to the current directory.
Example:
os.listdir('.') # Output: ['file1.txt', 'dir1', 'script.py']

os.mkdir(path)
Description: Creates a new directory at the specified path.
Example:
os.mkdir('new_folder')

os.makedirs(path)
Description: Recursively creates directories, including any necessary parent directories.
Example:
os.makedirs('folder1/folder2/folder3')

os.remove(path)
Description: Deletes the specified file.
Example:
os.remove('data.txt')

os.rmdir(path)
Description: Removes a single empty directory.
Example:
os.rmdir('old_folder')

os.removedirs(path)
Description: Removes directories recursively if they are empty.
Example:

os.removedirs('folder1/folder2')

2. Path Operations (with os.path)

os.path.join(path1, path2, ...)
Description: Joins one or more path components intelligently.
Example:
os.path.join('folder', 'file.txt') # Output: 'folder/file.txt'

os.path.exists(path)
Description: Returns True if the path exists.
Example:
os.path.exists('file.txt') # Output: True or False

os.path.isfile(path)
Description: Returns True if the path is a file.
Example:
os.path.isfile('report.pdf') # Output: True

os.path.isdir(path)
Description: Returns True if the path is a directory.
Example:
os.path.isdir('my_folder') # Output: True

os.path.abspath(path)
Description: Returns the absolute path of the given path.
Example:
os.path.abspath('script.py') # Output: '/home/user/project/script.py'

Datetime module

The datetime module provides classes for manipulating dates and times. It includes functions to work with both dates and timestamps.

import datetime

1. Getting Current Date and Time

datetime.datetime.now()
Description: Returns the current local date and time.
Example:

datetime.datetime.now()
# Output: datetime.datetime(2025, 6, 19, 14, 30, 45, 123456)

datetime.date.today()
Description: Returns the current local date.
Example:
datetime.date.today()
# Output: datetime.date(2025, 6, 19)

2. Creating Date, Time, and Datetime Objects

datetime.date(year, month, day)
Description: Creates a date object.
Example:
datetime.date(2025, 1, 1)
# Output: datetime.date(2025, 1, 1)

datetime.time(hour, minute, second)
Description: Creates a time object.
Example:
datetime.time(14, 30, 0)
# Output: datetime.time(14, 30)

datetime.datetime(year, month, day, hour=0, minute=0, second=0)
Description: Creates a datetime object.
Example:
datetime.datetime(2025, 6, 19, 14, 30)
# Output: datetime.datetime(2025, 6, 19, 14, 30)

3. Accessing Date/Time Components

For a datetime object dt:

dt = datetime.datetime.now()

dt.year # Output: e.g., 2025
dt.month # Output: e.g., 6
dt.day # Output: e.g., 19
dt.hour # Output: e.g., 14
dt.minute # Output: e.g., 30
dt.second # Output: e.g., 45

4. Formatting Dates and Times

strftime(format)
Description: Formats a datetime object to a string.
Example:

```
now = datetime.datetime.now()
now.strftime('%Y-%m-%d %H:%M:%S')
# Output: '2025-06-19 14:30:45'
```

Common format codes:
%Y – Year (e.g., 2025)

%m – Month (01 to 12)

%d – Day (01 to 31)

%H – Hour (00 to 23)

%M – Minute (00 to 59)

%S – Second (00 to 59)

5. Parsing Strings to Dates

datetime.datetime.strptime(date_string, format)
Description: Parses a string into a datetime object.
Example:

```
datetime.datetime.strptime('2025-06-19', '%Y-%m-%d')
# Output: datetime.datetime(2025, 6, 19, 0, 0)
```